CASCADE: Anytime Supernet Delivery for Federated Neural Architecture Search under Dynamic Bandwidth

Traditional Research Track

Abstract

Federated Learning (FL) has been increasingly adopted for its ability to train models across distributed clients without the data ever leaving the device it originated on. However, applying FL under data and resource heterogeneity means a single architecture rarely fits every client's compute and memory budget. Federated Neural Architecture Search addresses this by searching a family of architectures within the federated process itself. Existing methods, however, assume a client's uplink capacity stays fixed for the entire training run rather than accounting for how it fluctuates round by round. This assumption breaks down under real world connectivity, leading to high drop rates and accuracy loss in volatile edge deployments. To address this problem, we propose Communication Aware Supernet Chunking with Anytime Delivery (CASCADE), which decomposes a supernet update into nested shells and blocks so every prefix a deadline admits is itself a valid, smaller subnet. CASCADE is paired with MaxFit, an assignment policy that sizes each client's architecture to its current link capacity to avoid wasted computation. CASCADE cuts the share of clients losing their entire update from between 77 and 98% to under 1%. Accuracy rises at every deployment target, by 5.5 to 17.0% on CIFAR-10 and by 6.0% on CIFAR-100. MaxFit holds that accuracy for 22% fewer bytes and 43% less computation.

1 Introduction

Conventional machine learning models rely on centralizing training data on a central server. However, direct data centralization is increasingly untenable in domains governed by strict privacy regulations, proprietary concerns, and sensitive user telemetry. Federated Learning (FL) [1] addresses this challenge by decentralizing the optimization process across distributed clients. Instead of transmitting raw data, participants train locally and communicate only parameter updates to an orchestrating server. While initially adopted in institutional environments such as finance and healthcare, FL is increasingly deployed on resource-constrained edge and Internet of Things (IoT) devices that operate under strict compute, memory, and bandwidth limitations [2]. Predictive text input on mobile keyboards [3] and wake-word detection for smart-home voice assistants [4] already train on-device models this way, with a user's keystrokes or speech never leaving the device.

This deployment shift introduces a further constraint: no single architecture is optimal across such heterogeneous devices. A microcontroller and a flagship smartphone differ substantially in compute and memory budgets, rendering a fixed architecture either infeasible for constrained devices or wasteful for capable ones. Federated Neural Architecture Search closes this gap by searching a family of architectures within the federated process itself [5], enabling a single training pipeline to generate specialized models tailored to diverse hardware profiles and local data distributions.

However, implementing a federated architecture search on edge devices encounters an equally critical and often overlooked bottleneck in the communication channel. Although downlink speeds are typically sufficient to distribute parameters, edge networks are highly asymmetric, leaving client uplinks with much lower and less predictable throughput [6]. In standard synchronous training, the central server enforces a strict round deadline and discards any update that fails to complete in time. This makes optimization progress dependent on links that fluctuate drastically from round to round as devices move, share congested wireless channels, or switch network interfaces. Under such channel volatility, the clients' uplink becomes the primary point of failure for federated architecture search.

In this context of volatile bandwidth, this work enforces a fixed aggregation deadline to bound round duration and avoid stale models. To avoid discarding incomplete uploads at this deadline, the goal is to enable anytime partial delivery, allowing whatever fraction of an update arrives before the deadline to serve as a valid training contribution. This design avoids the standard all-or-nothing policy that drops late updates entirely, a practice especially damaging in non-IID edge networks where excluded clients often hold unique class

distributions found nowhere else in the federation.

Building on this premise, the central goal of this work is to train a single elastic supernet across edge clients experiencing volatile, time-varying bandwidth. Once trained, this supernet provides a complete family of specialized architectures matching diverse hardware deployment targets without repeating training per device class. Crucially, the hierarchical containment structure of the supernet itself provides the blueprint for anytime delivery, allowing partial updates to map directly onto coherent smaller submodels as network conditions fluctuate.

This work aims to answer the following research questions.

- How can a supernet update be transmitted so that a deadline-constrained round yields a usable contribution from every selected client, independently of any uplink capacity estimate and without systematically excluding those whose links are weakest under statistical data heterogeneity?
- How can round-by-round awareness of each client's uplink conditions reduce the computation a federated round expends on update content that is never delivered?

This work's main contributions are

- Communication-Aware Supernet Chunking with Anytime Delivery (CASCADE), a nested anytime delivery mechanism. It decomposes a supernet update into shells and blocks and streams them in an order in which every prefix the deadline admits is itself a valid, smaller subnet. The deadline alone determines the extent of each contribution, so no prior estimate of a client's capacity enters the decision. A region-specific aggregation rule accompanies the mechanism, correcting the bias coverage-count averaging incurs once clients reach different architecture regions at different long-run rates.
- MaxFit, a capacity-aware assignment policy that lowers the computation and communication a round consumes by matching each client's assigned ladder tier to the capacity its link reports that round, so that no client trains or transmits excessive content the deadline will discard.

2 Related Works

Prior literature intersecting this work falls into two categories: federated Neural Architecture Search (NAS) and communication-aware federated optimization. Existing approaches in either category do not address supernet optimization under a dynamic, deadline-bound uplink.

Table 1: Positioning against the closest related work.

	FedNAS [5] SuperFedNAS [7]	SALF [8]	FedAU / CSTAR-FL [9, 10]	**CASCADE**
Searches Arch.	✓			✓
Comm.-Aware		✓	✓	✓
Anytime Delivery		✓		✓
Agg. Correction			✓	✓
Pred.-Free				✓

Federated neural architecture search. Federated NAS decentralizes model search across edge participants to discover architectures tailored to edge constraints. Early methods such as FedNAS [5] distribute differentiable search (DARTS) across clients, treating federated rounds as logical synchronization barriers without accounting for transmission budgets or channel volatility. To reduce the prohibitive overhead of multi-model training, supernet-based methods adapt the once-for-all [11] paradigm. SuperFedNAS [7] trains a single elastic supernet from which specialized subnetworks are extracted after training. Subsequent extensions introduce resource awareness by sampling subnets based on client compute tiers [12, 13]. None of these methods survives a missed deadline: existing federated NAS methods assume deterministic uplink capacity and enforce all-or-nothing aggregation, so an update that fails to arrive before the round deadline is discarded entirely, with no mechanism to salvage a partial payload under channel degradation.

Communication- and resource-aware federated learning. A separate line of work adapts transmission to bandwidth constraints for a fixed architecture, at increasingly fine granularity. Coarsest, early approaches bind a submodel's capacity once to a client's declared hardware, often nested by design [14–18]. Closest to this work's own use of structural nesting is NeFL [19], which jointly scales depth and width and shows that shared weights across submodel tiers receive conflicting gradient updates, a dynamic elastic supernet training inherits by sharing weights the same way. Later methods dynamically resize a client's model or subnetwork assignment before each round, from an estimate of its current bandwidth [20–23], which remains fragile once a channel degrades unexpectedly within the round itself. At a finer granularity, SALF [8] transmits partial updates once the deadline expires, but streams gradients by layer index rather than by any structural containment relation, so a truncated delivery is an arbitrary parameter fragment rather than a functional submodel. This work instead aligns its transmission order with the containment already present among supernet tiers. Partial delivery also biases aggregation, and existing corrections are client-level only: FedAU [9] and CSTAR-FL [10] reweight a client's contribution by its participation frequency, but both apply the correction uniformly at the client level; because within-round partial delivery yields uneven coverage across regions of the architecture, not only across clients, this work

generalizes that correction into a region-specific one. None of these communication-aware methods performs architectural search, so none establishes the accuracy-versus-cost front this work's problem requires across heterogeneous deployment targets.

Table 1 summarizes this positioning. A method is comm.-aware if it adapts to a client's bandwidth or resource constraint; it delivers anytime if a deadline cut still yields a usable contribution; it corrects aggregation if it reweights unequal participation; it is prediction-free if no bandwidth estimate is required to decide what is deliverable. CASCADE bridges this divide by integrating supernet-based architecture search with within-round adaptive delivery and region-specific bias correction.

3 System Model

This section formalizes the federation, the elastic supernet search space, and the deadline-constrained, time-varying communication link; Figure 1 shows how they fit together.

3.1 Federation

The federation comprises K clients, indexed $i \in \{1,\dots,K\}$, each holding a private, non-identically distributed (non-IID) partition P_i of $n_i = |P_i|$ samples; raw samples never leave the device they originate on. Training spans R rounds, indexed $r \in \{1,\dots,R\}$. At the start of round r, the server selects an active subset $\mathcal{S}_r \subseteq \{1,\dots,K\}$ with $|\mathcal{S}_r| = \lceil C \cdot K \rceil$, broadcasts the current global parameters to $\mathcal{S}_r$, and each selected client trains on its own partition before the server aggregates whatever updates return before the round's deadline.

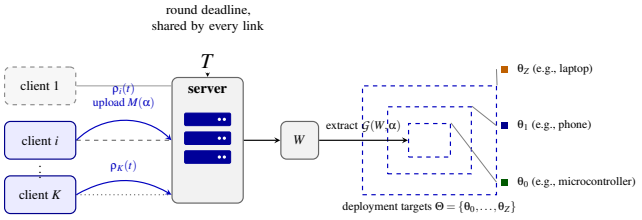


Figure 1: System overview. Dashed, unfilled boxes are deployment targets, nested to show they share one supernet's parameters. A straight line is a client's link; the curved arc over it is the message sent this round. A leader line and colored swatch label each target, from the smallest, θ_0, to the largest, θ_Z.

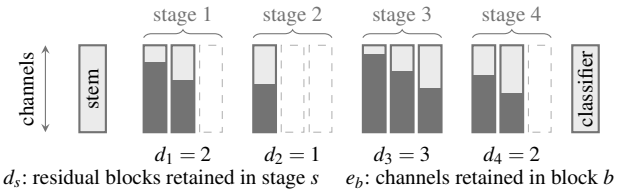


Figure 2: Elastic dimensions of the supernet backbone. A configuration $\alpha = (d, e)$ keeps a prefix of each stage's residual blocks, with dropped blocks drawn dashed, and a prefix of each block's channels, drawn shaded.

3.2 Elastic Supernet and Subnetwork Search Space

Heterogeneous edge platforms require different architectures after training, since a microcontroller and a laptop admit different compute and memory budgets, and no single architecture serves both without being infeasible for one or wasteful for the other. $Z+1$ is the number of deployment targets, and $\Theta = \{\theta_0,\dots,\theta_Z\}$ their inference-cost budgets. The federation's deliverable is one supernet, trained once, that serves every target in Θ without retraining per target.

The supernet formulation follows the once-for-all design [11] and its federated adaptation [7], whose elastic dimensions are those of a Residual Network (ResNet) backbone [24]. That backbone is a sequence of stages, each a sequence of residual blocks, in which depth varies by retaining a prefix of a stage's blocks and width by retaining a prefix of a block's channels (Figure 2). The supernet weights $W \in \mathcal{W}$ are the trainable parameters of the largest admissible network. An architecture configuration $\alpha = (d, e) \in \mathcal{A}$ pairs a per-stage depth vector $d \in \mathbb{Z}^S_{\geq 0}$ with a per-block expand-ratio vector $e \in [0,1]^B$, where S is the number of stages and B the number of residual blocks. The extraction operator $\mathcal{G} : \mathcal{W} \times \mathcal{A} \to \mathcal{W}$ selects the sub-tensor of W activated by α; write $W_\alpha = \mathcal{G}(W, \alpha)$. Because every $\alpha \in \mathcal{A}$ draws from the same underlying W, extracting a different architecture requires no retraining, distillation, or parameter transfer between architectures.

Two configurations are ordered, $\alpha \preceq \alpha'$, when every depth and expand-ratio component of α is no larger than α''s. This induces exact parameter containment, $\alpha \preceq \alpha' \implies \mathcal{G}(W, \alpha) \subseteq \mathcal{G}(W, \alpha')$, since a smaller depth and expand ratio only ever remove blocks and channels, never substitute them. The deployment targets are instantiated along that order, as $Z+1$ reference architectures $\alpha_0 \preceq \cdots \preceq \alpha_Z \in \mathcal{A}$, one per target, so $\mathcal{G}(W, \alpha_0) \subseteq \cdots \subseteq \mathcal{G}(W, \alpha_Z)$.

3.3 Communication Link

Every client i reaches the server over an uplink whose capacity $\rho_i(t)$ varies over time. Round r imposes a shared deadline T, so every client begins uploading at round r's start time t_r and any bits not received within T seconds are excluded from the round's aggregate, however many were in flight. This is a property of the deployment environment, not of any particular method. $\rho_i(t)$ fluctuates on the order of tens of seconds due to mobility, congestion, and hand-offs between radio technologies, and T is fixed by the application's tolerance for round latency. $M(\cdot)$ denotes the payload size in bits of a set of parameters, so $M(\alpha)$ abbreviates the payload of W_α.

4 Problem Formulation

This section formalizes training under the round deadline as a multi-objective optimization problem across the deployment targets Θ.

The local objective $L_i(w)$ is the loss parameters w achieve on partition P_i, so that training the supernet without any communication constraint minimizes

$$\min_W \mathbb{E}_{\alpha\sim\mathrm{Unif}(\mathcal{A})}\left[\sum_{i=1}^{K}\frac{n_i}{n}L_i(\mathcal{G}(W,\alpha))\right]. \tag{1}$$

Objective (1) is the training objective SuperFedNAS [7] solves, the anchor this work measures against throughout, and it assumes every client's update reaches the server in full, every round.

A round's deadline limits how much of a client's update actually arrives. Client i's deliverable volume in round r is $\kappa_{i,r} = \int_{t_r}^{t_r+T}\rho_i(t)\,dt$, the bits its link can carry before the deadline, and the delivery indicator $\delta_{i,r}(\alpha)\in\{0,1\}$ marks whether the architecture $\alpha_{i,r}\in\mathcal{A}$ client i trains that round fits: $\delta_{i,r}(\alpha)=1$ if $M(\alpha)\le\kappa_{i,r}$, else 0. Existing methods, including the anchor, treat this indicator as all-or-nothing – a client whose link cannot carry the whole payload contributes nothing to the round – so Objective (1) is optimized in practice not over the selected set $\mathcal{S}_r$ but over the delivered subset $\mathcal{S}_r^\delta=\{i\in\mathcal{S}_r:\delta_{i,r}(\alpha_{i,r})=1\}$, and $\mathcal{S}_r^\delta$ is not chosen; it is whichever clients the round's bandwidth realization happens to favor.

$\mu(\alpha)$ is the multiply-accumulate cost of α at inference; setting $\theta_z := \mu(\alpha_z)$ makes target z's budget exactly its reference architecture's cost. The deliverable of a federated NAS procedure across $Z+1$ heterogeneous targets is the vector $\big(L_{\mathrm{val}}(\mathcal{G}(W,\alpha_0)),\dots,L_{\mathrm{val}}(\mathcal{G}(W,\alpha_Z))\big)$, one entry per target, every entry a function of the same shared W. This is a multi-objective problem, not $Z+1$ independent ones. Because every α_z is extracted from the same W, a change that lowers loss for a large target z can raise it for a small target z', and the round-feasibility constraint below makes this tension concrete, since a larger α has a larger payload and is systematically less likely to satisfy $\delta_{i,r}(\alpha_{i,r})=1$ under a tight deadline, starving exactly the targets whose θ_z is large.

$$\min_W \quad \Big(L_{\mathrm{val}}(\mathcal{G}(W,\alpha_0)),\dots,L_{\mathrm{val}}(\mathcal{G}(W,\alpha_Z))\Big) \tag{2}$$

$$\text{s.t.} \quad W \text{ trained by (1) over } \mathcal{S}_r^\delta,\ \forall r\in\{1,\dots,R\} \tag{3}$$

Constraint (3) (*Round Feasibility*) restricts every round's contribution to the clients whose upload actually completed within the deadline, so the vector of achievable targets depends on the communication link, not only on the training procedure. Because Optimization Problem (2) minimizes a vector, no single W generally minimizes every target simultaneously. One W dominates another, W', when its loss is no larger at every z and strictly smaller at one. The solution of interest is therefore not a minimizer but the Pareto front, the set of loss vectors, achievable under Constraint (3), that no other achievable vector dominates. Nothing in the formulation forces loss to decrease as θ_z grows, since a supernet whose largest target performs worse than its smallest still satisfies every constraint above, and the resulting family is one no deployment would order by budget.

5 Method

5.1 Overview

CASCADE decomposes a client's update into nested pieces and streams them in an order in which every prefix the deadline admits is itself a valid, smaller subnet. The supernet supplies that order rather than the mechanism imposing one, since the containment property of Section 3.2 already ranks parameters by which subnets need them, so a stream cut short leaves the server holding a complete subnet of a lower tier instead of a fragment of a higher one. Section 5.2 defines the two objects a stream can be cut on, shells between tiers and blocks within them; Section 5.3 states the round procedure; Section 5.4 derives the aggregation correction partial arrivals require; Section 5.5 adds MaxFit, an assignment policy composed on top of the mechanism. Figure 3 gives the round CASCADE runs in three phases, and the subsections that follow return to each phase in turn.

5.2 Shells and Blocks

Delivering only a fraction of an update requires decomposing it into pieces that remain individually meaningful once the stream carrying them is interrupted. This subsection constructs two decompositions of the same coordinate set along independent axes: shells, which group coordinates by the tier of the ladder that first requires them, and blocks, which group them by the backbone's own structural units. Their intersection is the unit, the granularity at which a client transmits and the server accounts for what arrived.

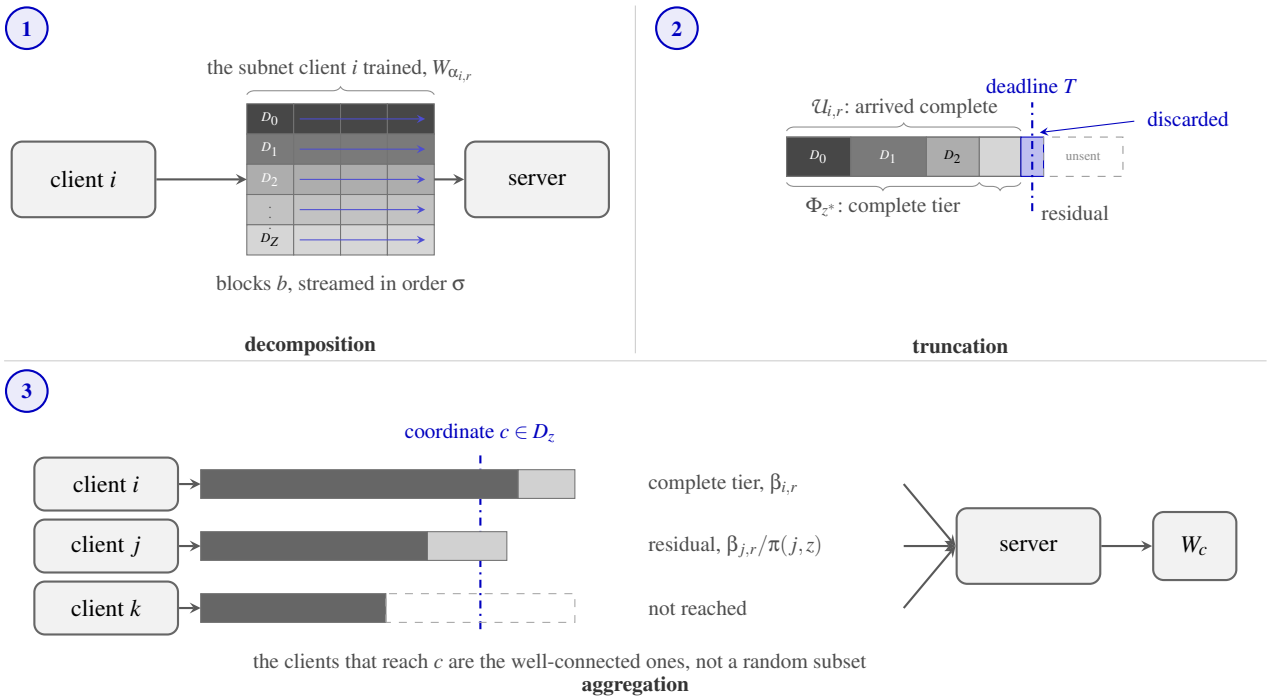


Figure 3: Nested anytime delivery, one round. Rows are shells D_0 (darkest, sent first) to D_Z (lightest); columns are blocks; a cell is a unit $u(z,b)$. The accent-colored unit is in flight when T hits; lighter cells past it were never attempted. The dash-dot line traces one coordinate c: client i's complete tier covers it, client j's residual unit covers it, client k's stream never reaches it.

The server holds the reference architectures $\alpha_0 \preceq \cdots \preceq \alpha_Z$ instantiating Θ as a ladder. A client trains an arbitrary $\alpha \in \mathcal{A}$, so its own ladder is the component-wise minimum of α against every reference point, $\alpha \wedge \alpha_z$. Φ_z denotes the coordinate indices of $W_{\alpha \wedge \alpha_z}$, the client's tier-$z$ subnet, and the z-th shell D_z the coordinates gained moving from tier $z-1$ to tier z, so $D_0 = \Phi_0$ and $D_z = \Phi_z \setminus \Phi_{z-1}$. The shells partition Φ_Z, and each prefix reassembles a tier exactly, $\bigcup_{j \le z} D_j = \Phi_z$, so a stream halting at the end of shell z leaves the server holding exactly Φ_z, a valid tier-z subnet rather than a fragment of a larger one.

Shell boundaries alone number $Z+1$ and are spaced unevenly, so an interruption between two tiers discards everything past the last tier completed, which may be most of a large shell. A block b is one of the B residual blocks of the backbone, and the unit $u(z,b)$ the coordinates of shell D_z belonging to block b (the stem and classifier form one further unit). Shells and blocks index the same coordinates independently, so an interruption is recoverable at every (z,b) pair rather than only at a tier boundary.

Units nest in width within a block, but are independent across blocks for aggregation. $u(z,b)$ extends the channels that $u(0,b),\ldots,u(z-1,b)$ carry and is meaningless without them, so a stream cannot skip ahead within a block; but averaging acts on one coordinate at a time, so block b's tier-z channels can be aggregated whether or not block b' delivered anything. They are not independent for deployment, since a tier-z subnet requires shell D_z complete in every block it spans. Separating what can be aggregated from what can be deployed is what admits the two aggregation paths of Section 5.3.

5.3 Nested Anytime Delivery

The transmission order σ is an enumeration of client i's units, admissible when it respects the within-block nesting of Section 5.2, so $u(z,b)$ precedes $u(z',b)$ whenever $z < z'$. Shell-major order sends every unit of shell D_0 across all blocks, then every unit of D_1, maximizing how many blocks reach a low shell before the deadline at the cost of the outer shells rarely completing; block-major order after a pivot follows shell-major order through that shell and then sends each block through its remaining shells in turn, letting a truncated tail complete a few blocks at full width instead of many at minimum width. Both are admissible, and everything the server does below is defined for any σ that is, so the choice between them is a design question this subsection returns to once the mechanism is in place.

A client transmits units in the order σ prescribes, so the number arriving complete is the longest prefix its link carries within the round,

$$m_{i,r} = \max\left\{ m \le |\mathcal{U}_i| \;:\; \sum_{k=1}^{m} M\big(u_{\sigma(k)}\big) \le \kappa_{i,r} \right\}. \qquad (4)$$

Algorithm 1: Nested anytime delivery, one round

Input: deadline T, ladder $\alpha_0 \preceq \cdots \preceq \alpha_Z$, admissible order σ
Output: $\{(\Phi^*_{i,r}, \mathcal{R}_{i,r})\}_{i \in \mathcal{S}_r}$

// Client side, every selected client i in parallel
for $i \in \mathcal{S}_r$ **in parallel do**
  $\mathcal{U}_{i,r} \leftarrow \emptyset$, $k \leftarrow 1$, $\tau \leftarrow 0$ // units enumerated by σ
  while $k \le |\mathcal{U}_i|$ **and** $\tau < T$ **do**
    $\tau \leftarrow \tau + \delta_i(u_{\sigma(k)})$ // $\delta_i(u)$: wall-clock time u takes over $\rho_i(\cdot)$
    if $\tau \le T$ **then** $\mathcal{U}_{i,r} \leftarrow \mathcal{U}_{i,r} \cup \{u_{\sigma(k)}\}$, $k \leftarrow k+1$
  // $u_{\sigma(k)}$ in flight when τ exceeds T is left incomplete, discarded
// Server side, once round r closes
for $i \in \mathcal{S}_r$ **do**
  $\mathcal{C}_{i,r} \leftarrow \bigcup_{u \in \mathcal{U}_{i,r}} u$ // coordinates the delivered units cover
  $z^*_{i,r} \leftarrow \max\{z \in \{0,\dots,Z\} : \bigcup_{j \le z} D_j \subseteq \mathcal{C}_{i,r}\}$ // (5): highest tier fully covered
  $\Phi^*_{i,r} \leftarrow \Phi_{z^*_{i,r}}$ // coordinates of that complete tier
  $\mathcal{R}_{i,r} \leftarrow \mathcal{U}_{i,r} \setminus \Phi^*_{i,r}$ // units delivered beyond it

$\mathcal{U}_{i,r} = \{u_{\sigma(k)} : k \le m_{i,r}\}$ denotes the units the server holds complete, and $\mathcal{C}_{i,r} = \bigcup_{u \in \mathcal{U}_{i,r}} u$ denotes the coordinates they cover; the unit in flight when the deadline expires is discarded, since a fraction of a unit extends no channel of any block.

The server classifies $\mathcal{C}_{i,r}$ against the ladder. The delivered tier is

$$z^*_{i,r} = \max\Big\{z \in \{0,\dots,Z\} \;:\; \bigcup_{j \le z} D_j \subseteq \mathcal{C}_{i,r}\Big\}, \qquad (5)$$

taken as -1 when shell D_0 is itself incomplete, with $\Phi_{-1} := \emptyset$. Write $\Phi^*_{i,r} := \Phi_{z^*_{i,r}}$ for client i's complete tier and $\mathcal{R}_{i,r} := \mathcal{U}_{i,r} \setminus \Phi^*_{i,r}$ for its residual units. $\Phi^*_{i,r}$ reassembles a valid tier-z^* subnet, aggregated exactly as an existing supernet method aggregates a complete update, at the anchor's own per-round weight $\beta_{i,r}$ [7] (defined in Section 5.4). The residual units complete no tier, but each is nonetheless a coordinate set the aggregation can average, since averaging acts on one coordinate at a time and σ delivers the inner units any delivered unit extends before it. Coordinates no client's stream reached this round stay untouched.

No step of this classification consults an estimate of $\rho_i(t)$. The deadline itself decides after the fact how far into the stream a client reached, so the feasibility indicator relaxes from an indicator on an entire architecture to one on a single coordinate c, $\delta_{i,r}(c) = \mathbf{1}[c \in \mathcal{C}_{i,r}]$ – a client's contribution degrades with its link instead of vanishing with it.

Algorithm 1 states the round procedure, executed by every participating client and by the server, and proceeds as follows. a) Every client's units are enumerated by σ, starting from shell D_0 (Line 2). b) The client advances through the stream while its transmission clock τ stays under the deadline T, which is what (4) counts (Line 3). c) The unit in flight when τ crosses T is left incomplete and never counted (Line 5). d) The server computes the delivered tier (5) for that client (Line 8). e) It splits the delivered set into the complete tier $\Phi^*_{i,r}$ (Line 9) and the residual units $\mathcal{R}_{i,r}$ beyond it (Line 10), aggregated under separate rules.

Everything above runs under any admissible σ, and that choice decides how deep a truncated round reaches. Depth here is the depth of the deepest *deployable* subnet, since a tier is covered only when its shell is complete in every block; measured that way, one order dominates every other.

Proposition 5.1 (Shell-major maximizes the delivered tier)**.** *Let σ be shell-major order, sending every unit of shell D_j before any unit of D_{j+1}. For any deliverable volume $\kappa_{i,r}$ and any order σ', admissible or not, the tier z^* that Algorithm 1 delivers under σ is at least the tier it delivers under σ'.*

Proof. Covering tier z means holding all of $\Phi_z = \bigcup_{j \le z} D_j$. The shells partition Φ_z, so covering it requires at least $\sum_{j \le z} M(D_j)$ bits under any order, and σ' covers tier z within $\kappa_{i,r}$ only if $\sum_{j \le z} M(D_j) \le \kappa_{i,r}$. Under σ those bits are the ones sent first, so whenever that inequality holds σ covers tier z as well. Since (5) returns the largest covered tier, the tier σ delivers is at least the tier σ' delivers. □

The bound covers tier depth and nothing else: an order is free to spend the same budget on fewer blocks at greater width, which Proposition 5.1 does not rank, and free to be cut with no tier covered at all, since admissibility constrains only the units within one block.

The reference architectures are nested, so a round that covers tier z contributes a complete update to targets 0 through z and to no others. The components of (2) a truncated round serves in full are therefore a prefix of the ladder, and Proposition 5.1 says shell-major's prefix contains that of every other order; every order delivers the same $\kappa_{i,r}$ bits, so the choice reallocates a fixed budget rather than enlarging it.

5.4 Aggregation under Partial Delivery

Under partial delivery a coordinate is no longer reached by every client in the round, and which clients those are changes from one coordinate to the next. The rule model-heterogeneous methods already apply to a partial submodel averages a coordinate over whichever clients reached it [14–16],

$$W_c \leftarrow \frac{1}{|\mathcal{K}_c|} \sum_{i \in \mathcal{K}_c} W_c^{(i)}, \qquad \mathcal{K}_c = \{i \in \mathcal{S}_r : c \in \mathcal{C}_{i,r}\}. \qquad (6)$$

$\mathcal{K}_c$ is not a uniform sample of the round's clients. Reaching coordinate c requires a client's stream to have travelled that far, so (6) averages over a subpopulation the bandwidth

Algorithm 2: Aggregation under partial delivery, one round

Input: $\{(\Phi^*_{i,r}, \mathcal{R}_{i,r})\}_{i\in\mathcal{S}_r}$ from Algorithm 1
Output: updated supernet weights W

1 **for** $i \in \mathcal{S}_r$ **do**
2 $w_{i,c} \leftarrow \beta_{i,r}\ \ \forall c \in \Phi^*_{i,r}$ // complete tier: anchor weight, uncorrected
3 **foreach** $u(z,b) \in \mathcal{R}_{i,r}$ **do**
4 $w_{i,c} \leftarrow \min\big(\beta_{i,r}/\pi(i,z), \bar{w}\big)\ \ \forall c \in u(z,b)$ // residual unit: corrected by (7)
5 $W_c \leftarrow \dfrac{\sum_{i\in\mathcal{K}_c} w_{i,c} W_c^{(i)}}{\sum_{i\in\mathcal{K}_c} w_{i,c}}\ \ \forall c$ with $\mathcal{K}_c \neq \emptyset$ // (8)
// coordinates no stream reached keep their previous value

selected. An inner shell is reached by nearly every client nearly every round, an outer one only by the best connected and rarely. Weighting each contribution by the inverse of the rate at which a client reaches the region moves the estimate back toward the population mean, the argument the client-level correction of [9] makes for participation. The long-run frequency with which client i's stream reaches shell D_z is $\pi(i,z) \in (0,1]$, estimated from the intervals between successive contributions rather than a plug-in probability, since the independence assumption behind that estimator does not hold for a link correlated round to round. The weight of client i's contribution at a coordinate c in shell D_z is

$$w_{i,c} = \min\left(\frac{\beta_{i,r}}{\pi(i,z)}, \bar{w}\right), \tag{7}$$

where $\beta_{i,r}$ is the client weight the anchor's own aggregation already assigns and $\bar{w}$ caps the correction, trading exact unbiasedness for stability where $\pi(i,z)$ is small enough that its inverse would amplify one client's noise past the worth of its data. The round then aggregates every touched coordinate, self-normalized,

$$W_c \leftarrow \frac{\sum_{i\in\mathcal{K}_c} w_{i,c} W_c^{(i)}}{\sum_{i\in\mathcal{K}_c} w_{i,c}}, \tag{8}$$

which recovers (6) exactly when every client in $\mathcal{K}_c$ carries the same weight. The frequency π is indexed by a shell as well as a client, separating this correction from [9, 10], which both reweight a client alone because participation is binary for them; anytime delivery instead makes participation partial, so one client is in for its inner shells and out for its outer ones in the same round.

Algorithm 2 separates the two paths Algorithm 1 hands it. A coordinate inside a client's complete tier takes the anchor weight $\beta_{i,r}$ uncorrected (Line 2); one reached only beyond that tier takes the corrected weight (7) (Line 4). A client that finished tier z this round is one for which those shells are routine, so $\pi(i,z)$ there is close to one and the correction on a complete prefix is near-identity; withholding it there also keeps the correction from compounding with $\beta_{i,r}$, since the client delivering the deepest tier is typically the one carrying the largest $\beta_{i,r}$ already. The residual units are, by construction, the part of the stream that arrived only because the client's link happened to permit it this round, so this is the population the correction exists for. This is worth stating plainly, since a coordinate in an outer shell can receive an uncorrected weight from one client and a corrected one from another, so (8) is not the uniform inverse-probability estimator that would make it unbiased – it upweights clients that reached a region marginally relative to those that reached it comfortably, moving the average toward the population without claiming to land on it.

5.5 MaxFit

Anytime delivery takes the delivery decision away from assignment, since the deadline classifies whatever arrives, whichever architecture the client was given, so sizing an assignment to a bandwidth estimate no longer protects a round against a wrong estimate. What assignment still decides is which architecture region a client spends its local computation on. Write $\tilde{\alpha}_{i,r} \in \mathcal{A}$ for the architecture the round assigns client i before any capacity is considered, and $\hat{\kappa}_{i,r} = \rho_i(t_{r-1})\,T$ for the server's estimate of its deliverable volume, formed by holding the capacity the link last showed. MaxFit caps the assignment at what the estimate admits,

$$\alpha_{i,r} = \begin{cases} \tilde{\alpha}_{i,r} & \text{if } M(\tilde{\alpha}_{i,r}) \le \hat{\kappa}_{i,r}, \\ \alpha_{\hat{z}} & \text{otherwise,} \end{cases} \tag{9}$$

where $\hat{z}$ is the largest $z \in \{0,\dots,Z\}$ with $M(\alpha_z) \le \hat{\kappa}_{i,r}$, and $\hat{z} = 0$ when no reference architecture fits.

Capping rather than selecting leaves the training distribution intact: a client whose estimate admits its assignment trains exactly what it was given, and only an assignment the link cannot carry is traded down. The policy acts on the tail of the distribution, not its centre.

MaxFit composes with CASCADE rather than replacing any part of it. The client streams whatever it was assigned in the order σ prescribes, and (4) still decides what arrives, so a misestimate in (9) costs how much of a client's computation reaches the server, never the correctness of what is aggregated: a wrong estimate spends computation, not accuracy. Capping down also narrows what a client trains, so the outer shells are trained by fewer clients than they would be without the cap, and Section 7 reports what that costs.

6 Experimental Setup

This work implements the decomposition of Section 5.2, the delivery mechanism of Section 5.3, the aggregation rule of

Section 5.4, MaxFit, and the communication simulator that enforces the round deadline. It builds them on the public release of SuperFedNAS [7], itself on FedML [25], and carries the anchor's supernet definition, sandwich sampler and aggregation schedule over unchanged, so that the two arms differ specifically in delivery.

6.1 Parameters

Every result is measured at one configuration (Table 2) unless it sweeps the quantity named on its horizontal axis: $K = 20$ clients of which $C \cdot K = 8$ participate per round, over $R = 150$ rounds under a deadline of $T = 20\,\mathrm{s}$, three seeds per arm at $\alpha_D = 100$, six at 0.5 and eight at 0.1. The server estimates capacity as $\hat{\kappa}_{i,r}$ from the previous round only.

Two departures from earlier idealizations are worth stating. First, the round charges upload time only, so a client's local computation falls outside T. A deployment that charged both against the same deadline would leave less of it for the upload, which makes the pressure reported here a lower bound on what such a deployment faces. Second, the sampler reserves one of the eight slots per round for the smallest reference architecture and one for the largest, drawing the remaining six freely – the anchor's own sandwich, kept so the comparison isolates delivery.

Every run orders units shell-major, sending all of D_0 before any unit of D_1 and so on. That order is what makes a delivered prefix decompose exactly into a complete tier and a residual, which is the split Section 5.3 classifies against; block-major order after a pivot is equally admissible, but it trades that guarantee for completing fewer blocks at greater width, and is not evaluated here.

The decomposition carries its own accounting, and it is small. A client's stream is at most $B(Z+1)+1$ units, 73 at the configuration of Table 2, each needing a shell index, a block index and a length; under a kilobyte of headers against a smallest payload of 41.6 MB, below 0.002 %. On the server, classifying a client costs one pass over the units it delivered.

Table 2: The configuration every arm is measured at.

Federation	
clients K / participating $C \cdot K$	20 / 8
rounds R / deadline T	150 / 20 s
local epochs / batch size	5 / 64
optimizer / learning rate	SGD / 0.1
Synthetic channel	
capacity states (Mbit/s)	50 / 10 / 2
profile shares, poor / avg / good	0.3 / 0.4 / 0.3
within-state spread s	0.45
Ladder, μ in GMAC / M in MB	
α_0, α_1	0.46 / 41.6, 0.96 / 98.6
α_2, α_3	1.64 / 137.6, 3.40 / 286.9
Aggregation	
β_r from / to / over	0.9 / 0.125 / 120 rounds
cap $\bar{w}$ / smoothing in $\hat{\pi}$	4 / Laplace 1

6.2 Channel

Three channels drive $\rho_i(t)$, one synthetic and two measured. The synthetic channel is a per-client Markov chain over three capacity states (50, 10, 2 Mbit/s); each client is drawn once into a poor/average/good connectivity profile (shares 0.3/0.4/0.3) that fixes its transition matrix and is kept for the whole run. Every profile is sticky, and the poor one is sticky in its worst state (0.70 against 0.30 for good), so drops correlate across rounds instead of being independent, and a deadline becomes a persistent handicap for some clients rather than a lottery every client loses equally often. Capacity is constant within a round and log-normally jittered across rounds inside a state (mean-preserving, $s = 0.45$, clipped to $\pm 2s$), which spreads out the reachable budgets without changing any state's mean rate.

The measured channel replays the LTE traces of Raca et al. [26] (135 recordings, two commercial Irish networks, five mobility patterns), read from the downlink since the traces carry no uplink capacity – an optimistic substitution, since downlink overstates what an uploading client gets. The third channel replays the Sprout traces [27], the only one of the three that measures the quantity the deadline actually constrains, since its collection kept a queue full in both directions at once, so a file records the rate the uplink permitted rather than the rate an application chose to send. What this channel costs is variety – eight drive tests from 2013, against the Irish archive's 135 recordings across five mobility patterns – which is why this work reports both channels rather than choosing.

Comparing the channels separates spread from level, and only the level is off. Log-throughput standard deviation is 1.38 synthetic against 1.15 measured (calibrated, slightly high), but median throughput is 20.3 Mbit/s synthetic against 6.5 Mbit/s measured downlink and 1.12 Mbit/s measured uplink. The synthetic channel is roughly three times too generous, which is why it is the mildest of the three and why it flatters the baselines rather than CASCADE.

6.3 Datasets

Experiments use CIFAR-10 and CIFAR-100, each 50,000 training and 10,000 test images. Training images are partitioned across the K clients by a Dirichlet distribution of concentration $\alpha_D \in \{100, 0.5, 0.1\}$, from near-uniform label proportions at 100 to partitions where a client holds few classes at 0.1; the subscript is unrelated to the architecture configuration α. The test set is not partitioned, and every accuracy in this document is measured on all 10,000 images.

6.4 Metrics

Every plotted quantity is a symbol built only from notation already introduced. Optimization Problem (2) minimizes test-set loss L_{val}; every number below is instead the accuracy that stands in for it.

- **Accuracy** $a(\alpha) \in [0,1]$: the fraction of the test set that subnet α classifies correctly, $a(\alpha) = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x,y)\in\mathcal{D}_{\text{test}}} \mathbf{1}[f(\mathcal{G}(W,\alpha),x) = y]$.
- **Target-averaged accuracy** $\bar{a} = \frac{1}{Z+1} \sum_{z=0}^{Z} a(\alpha_z)$; every reported accuracy is $\bar{a}$, an average over the family the federation ships. $\bar{a}_r$ denotes $\bar{a}$ at checkpoint round r; a final-round value averages the last five checkpoints (every 10 rounds), $\bar{a}_R = \frac{1}{5} \sum_{k=0}^{4} \bar{a}_{R-10k}$.
- **Front spread** $\Delta a = a(\alpha_Z) - a(\alpha_0) \in [-1,1]$: orders the targets by budget when positive; an inversion when negative. Compares targets within one W, unlike the dominance front of Section 4, which compares different W's at every target.
- **Drop rate** $\phi = \frac{1}{RCK} \sum_{r=1}^{R} \sum_{i\in\mathcal{S}_r} \mathbf{1}[\mathcal{C}_{i,r} = \emptyset] \in [0,1]$.
- **Transmitted volume** $V = \sum_{r=1}^{R} \sum_{i\in\mathcal{S}_r} \min(\kappa_{i,r}, M(\alpha_{i,r}))$, the total bits attempted.
- **Waste rate** $\omega \in [0,1]$: the fraction of V spent on the one unit each stream was truncated inside, $\omega = \frac{1}{V} \sum_{r,i} \left[\min(\kappa_{i,r}, M(\alpha_{i,r})) - \sum_{u\in\mathcal{U}_{i,r}} M(u)\right]$.
- **Training cost** $\mu(\alpha_{i,r})$: the inference-time MAC count applied to client i's assigned architecture, this work's proxy for training compute.

Accuracy figures show the mean over the seeds with the 95 % confidence interval of the mean shaded. Differences smaller than the run-to-run variation this setup produces at a fixed seed are not read as effects: an accidental pair of identical configurations diverged by 0.64 in $\bar{a}$ and 1.44 in Δa by the final round, which is the floor any claim here has to clear.

6.5 Baselines

Three references are run against CASCADE, and two choices inside it are isolated against the first.

- **SuperFedNAS** [7], the anchor, under its native all-or-nothing delivery: same codebase, seeds, sampler and sandwich, differing only in that an update whose last byte misses the deadline is discarded whole.
- **FedAvg** [1], a single ResNet-18 of 44.7 MB, trained without architecture search.
- **FedNAS** [5], a DARTS supernetwork of 201.9 MB, searched differentiably across clients.

Neither fixed-architecture reference ships a family, so their drop rates are not comparable slot for slot with those reported here. FedNAS enters the delivery metrics but not the accuracy ones, since both its deadline-bound and unconstrained runs remained at chance at every evaluation, identifying the optimization setup rather than the channel as the cause; a DARTS mixture also has no nested subset structure, so anytime delivery has nothing to truncate to. Two components of CASCADE are varied against the anchor rather than against a competitor: **decomposition**, shells against blocks (Section 5.2), and **assignment**, the blind sampler against MaxFit (Section 5.5) over one unchanged delivery mechanism.

7 Results Analysis

This section evaluates CASCADE along three axes: how much of a round's contribution the deadline actually delivers, the accuracy that delivery preserves across the deployment ladder, and the cost at which MaxFit sustains it. Every figure reports means at the configuration of Table 2 unless it sweeps the quantity named on its horizontal axis.

Figure 4 sweeps the deadline the rest of this section holds fixed. **Every arm that ships an atomic update degrades as T tightens; CASCADE over blocks does not.** It loses no client across the whole range, where the anchor loses 77.2 % of slots at the operating point. **Nesting alone does not explain this.** The shell decomposition nests identically yet still drops 48.0 %, because its innermost shell is already too large for a weak link to finish. **The mechanism does not buy its accuracy with extra bandwidth.** Both arms attempt the same volume of bytes and differ only in how much of it the server can use (86.5 against 36.4 %). **Measured channels confirm the direction, and worsen every magnitude.** On the Irish downlink the anchor drops 95.8 % of slots against CASCADE's 0.8; on the Sprout uplink it drops 97.9 % against 0.5, at the cost of more waste as slower links truncate more streams mid-unit.

7.1 What discarding costs

A drop rate is not yet an accuracy cost, because Section 6.2's connectivity profiles are sticky: the clients that miss the deadline are largely the same every round, so what the server never sees depends on whether those clients hold anything the rest do not.

Figure 5 shows the gap is larger under skewed data. CASCADE is ahead of the anchor at every α_D, by +5.48 points at $\alpha_D = 100$ against +16.96 and +15.20 at $\alpha_D = 0.5$ and 0.1: at low heterogeneity an excluded client is substitutable, but under skew persistent exclusion removes classes from the federation rather than merely reducing its data.

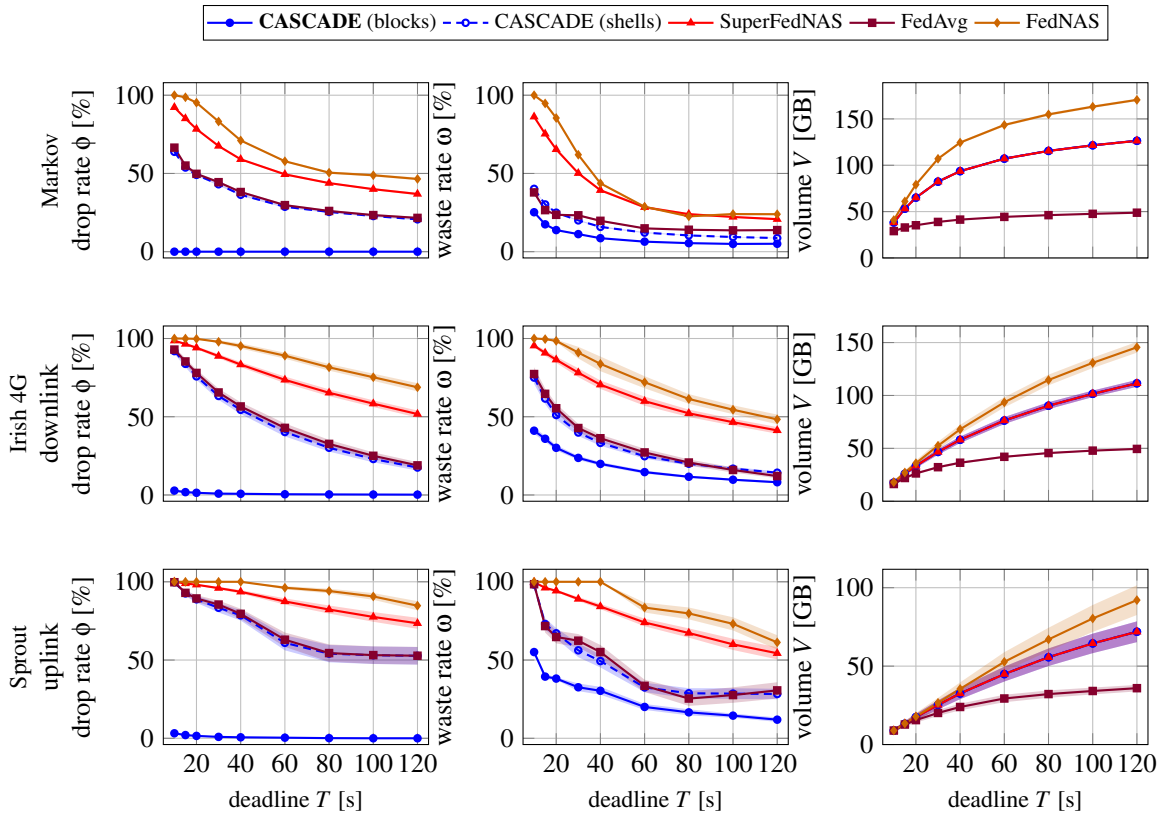


Figure 4: Drop rate ϕ, waste rate ω and transmitted volume V against the round deadline, on the three channels of Section 6.2. Lines show the mean over 20 channel seeds on each channel; bands are 95% confidence intervals of the mean.

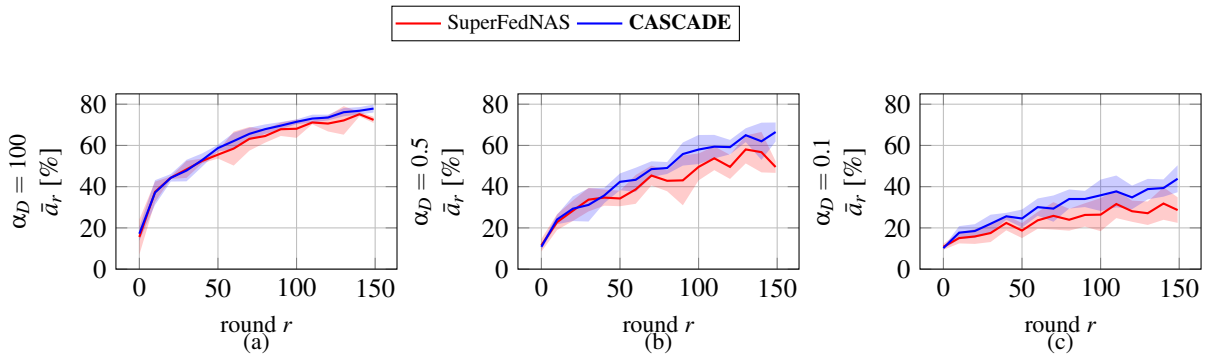


Figure 5: Accuracy of CASCADE against SuperFedNAS's own all-or-nothing delivery over the full canonical run (round deadline $T = 20$ s), at three non-IID degrees α_D: (a) 100, (b) 0.5, (c) 0.1. Mean over the seeds with the 95 % confidence interval of the mean shaded: three seeds at (a), six at (b), eight at (c). Lower α_D is more non-IID. The three panels share one vertical scale.

The advantage does not trade one target for another. CASCADE dominates the anchor at every one of the four deployment targets, at every non-IID degree, satisfying the dominance relation Section 4 defines coordinate-wise rather than on average.

Under a correlated partition, persistent exclusion is a bias and not a subsample. Non-IID partitioning gives each client a label distribution of its own, so which clients are excluded decides what the federation is never shown. Connectivity is rarely independent of that distribution: device class, region and access technology all vary with the data being generated, so the clients a deadline excludes hold a particular kind of data rather than an arbitrary sample of it. The exclusion is then systematic, and an accuracy averaged over classes conceals it, since a class learned poorly and a class never learned enter that average alike while differing entirely for a

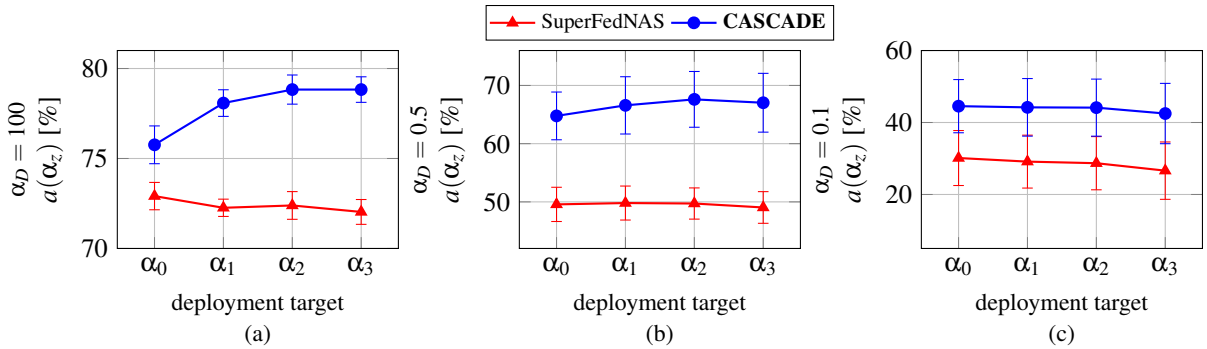


Figure 6: Accuracy across the deployment ladder, at each of its four reference architectures, for both arms at three non-IID degrees α_D: (a) 100, (b) 0.5, (c) 0.1. A rising line has the ordering a ladder is meant to have; a flat or falling one has lost it. Markers are the mean over the seeds (three at (a), six at (b), eight at (c)) and bars their standard deviation.

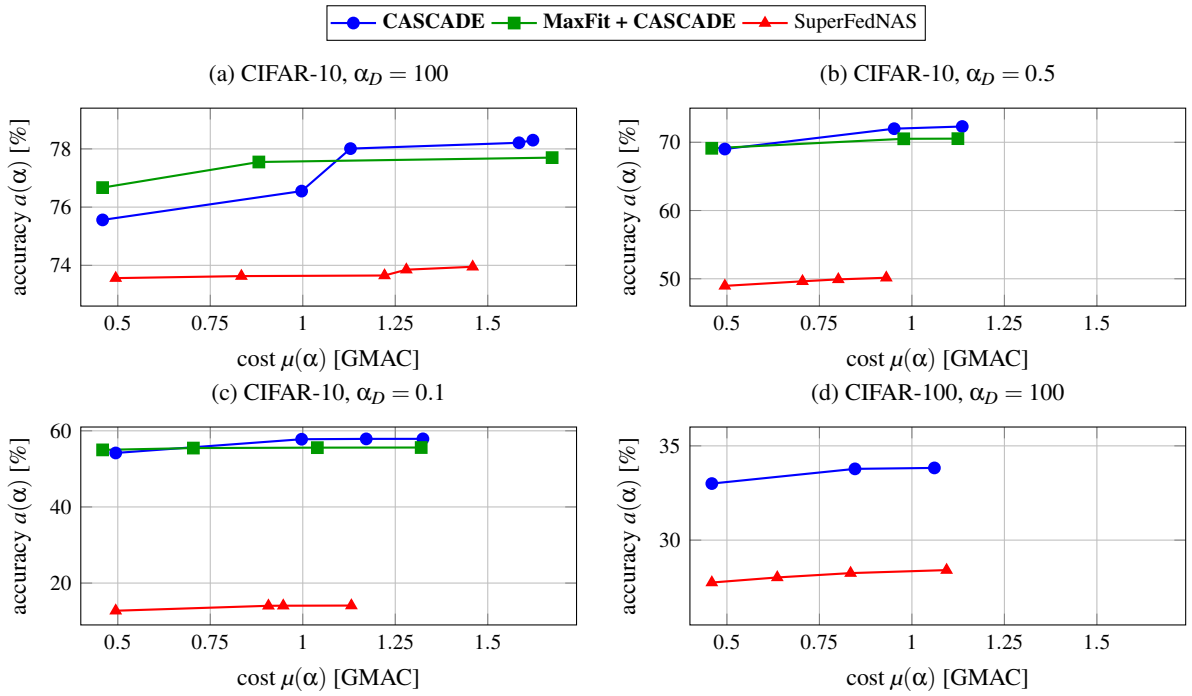


Figure 7: The Pareto front: accuracy against inference cost, for the architectures the evolutionary search returns that no cheaper architecture beats. Both axes increase by construction: a budget admits every cheaper architecture, so a front cannot fall as cost rises. Cost is on the same scale in all four panels (a)–(d) and accuracy is not, the conditions differing by fifty points. Single seed; MaxFit appears on CIFAR-10, where it has a trained supernet.

deployment that must serve both. As the partition hardens, the same drop rate removes a larger share of what is held nowhere else, which is the widening reported above.

CASCADE acts on this at the level of participation: every selected client contributes whatever its link admits, so no client's data is structurally absent from a round. Partial delivery then raises a second form of the same bias, which is what the region-specific correction of Section 5.4 exists for. Clients reach the inner shells at comparable rates and the outer shells at rates their links decide, so an average over whichever clients arrived would estimate the outer regions of the supernet from the best connected among them. Correcting delivery without correcting aggregation would reinstate across regions of the architecture the bias the mechanism removes across clients.

Figure 6 reports accuracy at all four rungs of the deployment ladder, at each non-IID degree. The front spread Δa (Section 6.4) is the rise from α_0 to α_3.

At $\alpha_D = 100$ the anchor's front is inverted. Its largest architecture is also its largest payload, so under all-or-nothing delivery it is the least trained region of the supernet, and the ordering the ladder is supposed to have collapses (spread -0.88). CASCADE restores it ($+3.07$), concentrated at the top of the ladder exactly where the argument predicts: $+6.80$ at α_3 against $+2.84$ at α_0.

7.2 Architectures under a cost budget

This subsection asks what a deployment actually gets at a given cost budget, rather than at the $Z+1$ fixed targets Section 7.1 reports. Figure 7 searches each trained supernet for the best architecture under a sequence of inference-cost budgets, and reduces what it finds to the non-dominated set: the Pareto front, not a single scalarized number.

Unlike the spread of Section 7.1, this front cannot invert by construction: a budget's feasible set contains every cheaper one, so the best attainable accuracy is non-decreasing in the budget, and an architecture costing more without scoring better is simply dominated.

The gain holds at every cost, and the fronts do not overlap. An architecture from CASCADE's supernet outscores anything the anchor's offers by $+2.00$ to $+4.35$ points at $\alpha_D = 100$, widening to $+41.45$ to $+43.80$ at $\alpha_D = 0.1$; CASCADE's cheapest architecture beats the anchor's costliest at every degree. **The anchor's front is nearly flat; CASCADE's is not.** It rises 0.39 points across the whole sweep at $\alpha_D = 100$, against CASCADE's 2.74, because its expensive architectures are the least trained region of its supernet under all-or-nothing delivery, and a search allowed to spend more finds nothing there worth having. MaxFit trades some of this rise for delivering it at lower cost: it starts at or above CASCADE but climbs less steeply, consistent with training the small architectures well and the large ones rarely.

The separation holds on CIFAR-100. CASCADE's front runs 33.00–33.83 against the anchor's 27.75–28.41, a gap that widens to seven points on a second seed. Architecture selection uses a ten-batch proxy, with accuracies reported on the held-out test set; no search reaches its cost ceiling at $R = 150$ rounds; and at $\alpha_D = 0.1$, where the figure draws one seed, three seeds separate the fronts by a mean of 24.66 points, none below 10.66, although the mean accuracy gap over the eight trained seeds is 15.20.

7.3 MaxFit

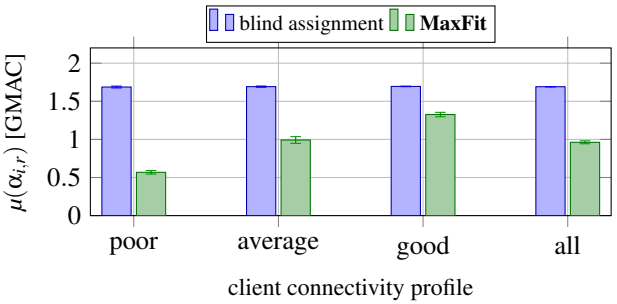


Figure 8: Local training cost per client-round under blind assignment and under MaxFit, by the client's connectivity profile. Bars are the mean over three seeds and whiskers their standard deviation.

MaxFit composes on top of CASCADE on the subnet assignment layer, and its result is not a further accuracy gain. **Relative to CASCADE under blind assignment, it holds accuracy for 22.1 % fewer bytes.** At $\alpha_D = 0.1$ three further seeds give 22.5 % fewer bytes with the accuracy difference indistinguishable from zero.

Figure 8 shows why: blind assignment spends local compute uniformly regardless of a client's link, while MaxFit concentrates the saving where it costs least, cutting a poorly connected client's training by 66.3 % against 21.8 % for a well connected one, compute that was largely unusable anyway since it belonged to clients whose deadline arrives before their stream ends. **The saving is not completely free.** MaxFit trains the large targets more rarely. Since the largest reference architecture needs more sustained throughput than any channel state here supplies, its front spread ($+0.69$) recovers less of the anchor's inversion than CASCADE's ($+3.07$), but keeps it positive.

8 Conclusion

This work addressed the assumption, implicit in federated architecture search, that a client's update always returns within the round, an assumption real connectivity breaks. CASCADE removes it: it decomposes a supernet update into nested units, streams them so that whatever the deadline admits is itself a valid smaller subnet, and aggregates each coordinate over the clients whose stream actually reached it, with MaxFit composing on top to size each client's architecture to its link.

Three findings stand out. Granularity is what keeps a client contributing under a tight deadline. Failed delivery does not merely cost accuracy, it removes the ordering a supernet exists to provide, so a larger architecture stops being a better one. And the advantage is larger under statistical heterogeneity, $+5.48$ points at $\alpha_D = 100$ against $+15.20$ at $\alpha_D = 0.1$.

Three questions remain open. The first is whether the decomposition generalizes past a block-structured backbone, such as more complex neural network architectures. The second is whether an order chosen rather than fixed is worth the choosing: shells and blocks nest identically and differ only in where a stream may be cut, yet one loses half a round at the operating point and the other loses none, so the same lever is there for block-major order after a pivot, which completes fewer blocks at greater width exactly where the outer shells are thinnest. The third is whether anytime delivery and asynchronous aggregation compose into a single policy, since both trade completeness against speed, one within a round and one across them, and neither has been asked to do it with the other.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.

[2] D. Nguyen, M. Ding, P. Pathirana, A. Seneviratne, J. Li, and H. Poor, "Federated Learning for Internet of Things: A Comprehensive Survey," *IEEE Communications Surveys & Tutorials*, 2021.

[3] Z. Xu, Y. Zhang, G. Andrew, C. A. Choquette-Choo, P. Kairouz, H. B. McMahan, J. Rosenstock, and Y. Zhang, "Federated Learning of Gboard Language Models with Differential Privacy," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Industry Track)*, 2023.

[4] D. Leroy, A. Coucke, T. Lavril, T. Gisselbrecht, and J. Dureau, "Federated Learning for Keyword Spotting," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019.

[5] C. He, M. Annavaram, and S. Avestimehr, "Towards Non-I.I.D. and Invisible Data with FedNAS: Federated Deep Learning via Neural Architecture Search," 2020.

[6] R. Hayek, K. Comer, J. Chung, C. Murthy, R. Kettimuthu, and I. Kadota, "Beyond Assumptions: Measuring Federated Learning over Real 5G Networks," 2025.

[7] A. Khare, A. Agrawal, A. Annavajjala, P. Behnam, M. Lee, H. Latapie, and A. Tumanov, "SuperFedNAS: Cost-Efficient Federated Neural Architecture Search for On-Device Inference," in *European Conference on Computer Vision (ECCV)*, 2024.

[8] N. Lang, A. Cohen, and N. Shlezinger, "Stragglers-Aware Low-Latency Synchronous Federated Learning via Layer-Wise Model Updates," 2024.

[9] S. Wang and M. Ji, "A Lightweight Method for Tackling Unknown Participation Statistics in Federated Averaging," in *International Conference on Learning Representations (ICLR)*, 2024.

[10] X. Xu, A. Di Maio, E. Samikwa, and T. Braun, "CSTAR-FL: Stochastic Client Selection for Tree All-Reduce Federated Learning," *IEEE Transactions on Mobile Computing*, 2025.

[11] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-All: Train One Network and Specialize it for Efficient Deployment," in *International Conference on Learning Representations (ICLR)*, 2020.

[12] A. Yang and Y. Liu, "Heterogeneity-Aware Personalized Federated Neural Architecture Search," *Entropy*, vol. 27, no. 7, p. 759, 2025.

[13] B. Khan and M. Daneshtalab, "DeepFedNAS: Efficient Hardware-Aware Architecture Adaptation for Heterogeneous IoT Federations via Pareto-Guided Supernet Training," 2026.

[14] E. Diao, J. Ding, and V. Tarokh, "HeteroFL: Computation and Communication Efficient Federated Learning for Heterogeneous Clients," in *International Conference on Learning Representations (ICLR)*, 2021.

[15] S. Alam, L. Liu, M. Yan, and M. Zhang, "FedRolex: Model-Heterogeneous Federated Learning with Rolling Sub-Model Extraction," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[16] S. Horvath, S. Laskaridis, M. Almeida, I. Leontiadis, S. Venieris, and N. Lane, "FjORD: Fair and Accurate Federated Learning under Heterogeneous Targets with Ordered Dropout," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[17] F. Ilhan, G. Su, and L. Liu, "ScaleFL: Resource-Adaptive Federated Learning with Heterogeneous Clients," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[18] L. Yi, H. Yu, C. Ren, G. Wang, X. Liu, and X. Li, "Federated Model-Heterogeneous Matryoshka Representation Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[19] H. Kang, S. Cha, J. Shin, J. Lee, and J. Kang, "NeFL: Nested Model Scaling for Federated Learning with System Heterogeneous Clients," 2025.

[20] P. Li, G. Cheng, X. Huang, J. Kang, R. Yu, Y. Wu, and M. Pan, "AnycostFL: Efficient On-Demand Federated Learning over Heterogeneous Edge Devices," in *IEEE International Conference on Computer Communications (INFOCOM)*, 2023.

[21] N. Emami, A. Di Maio, and T. Braun, "FedForce: Network-Adaptive Federated Learning for Reinforced Mobility Prediction," in *IEEE Conference on Local Computer Networks (LCN)*, 2023.

[22] D. Yao, "Exploring System-Heterogeneous Federated Learning with Dynamic Model Selection," 2024.

[23] E. Samikwa, A. Di Maio, and T. Braun, "ARES: Adaptive Resource-Aware Split Learning for Internet of Things," *Computer Networks*, 2022.

[24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[25] C. He, S. Li, J. So, M. Zhang, H. Wang, X. Wang, P. Vepakomma, A. Singh, H. Qiu, L. Shen, P. Zhao, Y. Kang, Y. Liu, R. Raskar, Q. Yang, M. Annavaram, and S. Avestimehr, "FedML: A Research Library and Benchmark for Federated Machine Learning," 2020.

[26] D. Raca, J. Quinlan, A. Zahran, and C. Sreenan, "Beyond Throughput: A 4G LTE Dataset with Channel and Context Metrics," in *ACM Multimedia Systems Conference (MMSys)*, 2018.

[27] K. Winstein, A. Sivaraman, and H. Balakrishnan, "Stochastic Forecasts Achieve High Throughput and Low Delay over Cellular Networks," in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2013.

A Appendix

A.1 Notation Reference

Table 3 collects the symbols used throughout this document, for reference.

A.2 Replications

The figures report means over replications; this appendix gives the replications themselves, so that a claim in the results can be checked against the runs rather than taken. Every number here is read from the same file the corresponding figure is drawn from.

Table 4 and Table 5 carry the accuracy runs on the two datasets. Table 6 carries delivery at the operating point across the three channels, Table 7 the local training cost by connectivity profile, and Table 8 the endpoints of every searched front.

Table 3: Key notation used throughout this document.

Notation	Definition
K, C, R	number of clients, participation fraction, rounds
n_i, n, $L_i(w)$	size of client i's partition, total samples, its local loss
α_D	Dirichlet concentration of the label partition
$\mathcal{S}_r$, $\mathcal{S}_r^{\delta}$	round-r active subset ($\lvert\mathcal{S}_r\rvert = \lceil C \cdot K \rceil$), its delivered subset
$\mathcal{W}$, W, α, $\mathcal{A}$	parameter space, supernet weights, architecture, architecture space
d, e, S, B	depth vector, expand-ratio vector, stages, blocks
$\mathcal{G}(W,\alpha)$, W_α	subnet extraction operator, extracted parameters
Z, Θ	number of targets minus one, deployment target budgets $\{\theta_0,\ldots,\theta_Z\}$
θ_z, α_z	target z's budget, its reference architecture
$\mu(\alpha)$, $L_{\text{val}}(w)$	inference cost of α in MACs, test-set loss of w
$\rho_i(t)$, T, t_r	client i's uplink capacity, round deadline, round r's start time
$M(\alpha)$	payload size of $\mathcal{G}(W,\alpha)$, in bits
$\hat{\kappa}_{i,r}$, $\hat{z}$	estimated deliverable volume, the tier MaxFit caps to
$\tilde{\alpha}_{i,r}$	architecture assigned to client i before capacity is considered
$\kappa_{i,r}$, $\delta_{i,r}(\alpha)$, $\alpha_{i,r}$	client i's deliverable volume in round r, delivery indicator, its trained architecture
Φ_z, D_z	tier-z parameter set, tier-z shell
σ, $\mathcal{U}_i$	transmission order, client i's set of units
s	within-state spread of the synthetic channel
$m_{i,r}$, $\mathcal{U}_{i,r}$	units client i completes in round r, the set they form
$\mathcal{C}_{i,r}$	coordinates those units cover
$z^*_{i,r}$	delivered tier
$\Phi^*_{i,r}$, $\mathcal{R}_{i,r}$	client i's complete tier, its residual units
β_r, $\beta_{i,r}$	round-r share of the largest-subnet client, share client i gets
b, $u(z,b)$	architecture block, unit (shell z of block b)
$\pi(i,z)$	long-run frequency client i reaches shell D_z
$w_{i,c}$, $\bar{w}$	aggregation weight of client i at coordinate c, its cap
$\mathcal{K}_c$	clients whose stream reached coordinate c this round
$W_c^{(i)}$	client i's locally trained value at coordinate c
$a(\alpha)$, $\bar{a}$, $\bar{a}_r$	accuracy of subnet α, target-averaged accuracy, its value at checkpoint round r
Δa	front spread, $a(\alpha_Z) - a(\alpha_0)$
ϕ, ω	drop rate, waste rate
V	transmitted volume, in gigabytes

Table 4: Accuracy replications behind Figure 5 and Figure 6. One row per seed; $\bar{a}$ columns are the last checkpoint and the mean of the last five. The figures plot these means and the 95% confidence interval across the seeds. A dagger marks the $\alpha_D = 0.1$ seed-0 pair, kept here and left out of every mean and figure: its CASCADE run used the earlier completeness aggregation rule where every other CASCADE run used the cascade rule, and dropping one arm of the pair would leave the arms with different n.

α_D	arm	seed	final	mean_5	α_0	α_3
100	SuperFedNAS	0	73.10	72.65	73.59	72.77
		1	72.02	72.27	72.09	71.92
		2	72.08	71.92	73.06	71.41
	mean		72.40	72.28	72.91	72.03
	CASCADE	0	77.35	75.79	75.56	78.15
		1	78.78	75.76	76.89	79.57
		2	77.49	74.85	74.82	78.77
	mean		77.87	75.47	75.76	78.83
0.5	SuperFedNAS	0	49.12	55.14	48.53	48.85
		1	54.07	52.58	53.73	53.57
		2	46.56	53.32	46.56	46.46
		3	50.66	55.06	50.14	50.86
		4	49.66	55.05	51.99	47.29
		5	47.13	50.05	46.48	47.22
	mean		49.53	53.53	49.57	49.04
	CASCADE	0	70.25	67.23	68.57	70.19
		1	69.62	63.07	70.14	69.46
		2	61.37	60.26	60.29	61.44
		3	69.06	61.63	62.03	72.27
		4	60.39	63.02	61.40	59.97
		5	68.29	59.24	66.16	68.85
	mean		66.50	62.41	64.77	67.03
0.1	SuperFedNAS	$0^\dagger$	12.74	33.88	11.86	12.28
		1	17.82	26.21	18.01	17.13
		2	34.66	34.88	34.62	34.58
		3	19.27	26.15	21.53	15.54
		4	28.68	26.12	28.90	28.22
		5	27.06	28.90	27.15	26.98
		6	37.26	31.54	38.08	34.63
		7	36.80	34.70	39.24	34.98
		8	27.61	27.09	33.52	20.94
	mean		28.64	29.45	30.13	26.62
	CASCADE	$0^\dagger$	54.97	52.32	53.05	56.40
		1	40.73	33.02	41.76	37.66
		2	46.42	43.70	45.85	46.96
		3	52.51	35.86	50.56	52.89
		4	29.18	31.95	29.51	28.39
		5	54.49	45.31	54.87	53.31
		6	40.85	39.80	44.29	38.47
		7	43.69	40.02	46.12	42.50
		8	42.88	41.52	43.37	39.71
	mean		43.84	38.90	44.54	42.49

Table 5: Accuracy replications on CIFAR-100 at $\alpha_D = 100$. Same extraction as Table 4.

arm	seed	final	mean_5	α_0	α_3
SuperFedNAS	0	27.53	29.91	27.75	26.52
	1	26.08	28.77	26.68	25.08
	2	29.56	30.92	30.35	28.85
mean		27.73	29.87	28.26	26.82
CASCADE	0	32.90	33.02	33.00	32.45
	1	33.98	31.81	33.93	33.65
	2	34.44	33.06	34.61	33.84
mean		33.77	32.63	33.85	33.31

Table 6: Delivery at the operating point $T = 20\,\mathrm{s}$ on all three channels, behind Figure 4. The synthetic channel is replicated over three channel seeds; the two measured channels are single traces. ϕ is the drop share, ω the wasted share of the bytes sent, and V the total attempted volume.

		Synthetic			Measured	
arm	metric	seed 0	seed 1	seed 2	Irish 4G	Sprout
CASCADE (blocks)	ϕ [%]	0.0	0.0	0.0	0.8	0.5
	ω [%]	13.5	14.3	12.3	34.5	38.2
	V [GB]	65.8	62.8	64.0	30.2	15.3
CASCADE (shells)	ϕ [%]	48.0	52.4	49.8	78.6	90.8
	ω [%]	24.6	26.0	23.8	59.6	68.1
	V [GB]	65.8	62.8	63.9	30.2	15.3
SuperFedNAS	ϕ [%]	77.2	78.7	78.3	95.8	97.9
	ω [%]	63.6	63.8	64.6	91.3	92.7
	V [GB]	65.8	62.8	63.9	30.2	15.3
FedAvg	ϕ [%]	48.9	52.8	50.7	80.9	91.3
	ω [%]	23.1	26.2	22.7	60.6	66.5
	V [GB]	35.7	34.3	34.3	26.0	13.9
FedNAS	ϕ [%]	95.0	96.3	95.4	100.0	100.0
	ω [%]	84.9	88.3	85.8	100.0	100.0
	V [GB]	80.3	75.6	78.3	31.6	15.7

Table 7: Local training cost per client-round in GMAC, behind Figure 8. One column per seed.

assignment	profile	seed 0	seed 1	seed 2	mean
blind	poor	1.686	1.676	1.699	1.687 ± 0.012
	average	1.694	1.699	1.684	1.692 ± 0.008
	good	1.694	1.699	1.693	1.695 ± 0.003
	all	1.692	1.692	1.692	1.691 ± 0.000
MaxFit	poor	0.589	0.543	0.574	0.568 ± 0.023
	average	1.027	1.007	0.943	0.992 ± 0.044
	good	1.312	1.307	1.360	1.326 ± 0.029
	all	0.984	0.946	0.958	0.962 ± 0.019

Table 8: Endpoints of every searched front behind Figure 7: the cheapest and dearest non-dominated architecture, and the accuracy between them.

condition	arm	seed	points	cheapest	dearest	*a* cheapest	*a* dearest
CIFAR-10, $\alpha = 100$	SuperFedNAS	0	5	0.49	1.46	73.56	73.95
		1	4	0.49	1.14	72.10	73.00
	CASCADE	0	5	0.46	1.62	75.56	78.30
		1	5	0.49	1.93	76.76	79.46
	MaxFit + CASCADE	0	3	0.46	1.67	76.67	77.70
		1	2	0.46	0.96	77.80	78.06
CIFAR-10, $\alpha = 0.5$	SuperFedNAS	0	4	0.49	0.93	48.96	50.15
		1	2	0.49	0.89	53.03	55.67
	CASCADE	0	3	0.49	1.14	69.01	72.31
		1	4	0.46	1.27	70.14	71.20
	MaxFit + CASCADE	0	3	0.46	1.12	69.12	70.53
CIFAR-10, $\alpha = 0.1$	SuperFedNAS	0	4	0.49	1.13	12.73	14.09
		1	5	0.46	1.02	18.01	19.23
		2	3	0.49	1.21	34.64	35.19
	CASCADE	0	4	0.49	1.32	54.18	57.89
		1	3	0.49	0.84	42.45	44.07
		2	5	0.46	1.28	45.85	47.42
	MaxFit + CASCADE	0	4	0.46	1.32	54.99	55.61
CIFAR-100, $\alpha = 100$	SuperFedNAS	0	4	0.46	1.09	27.75	28.41
		1	2	0.49	0.70	26.79	26.82
	CASCADE	0	3	0.46	1.06	33.00	33.83
		1	2	0.46	1.00	33.93	34.68